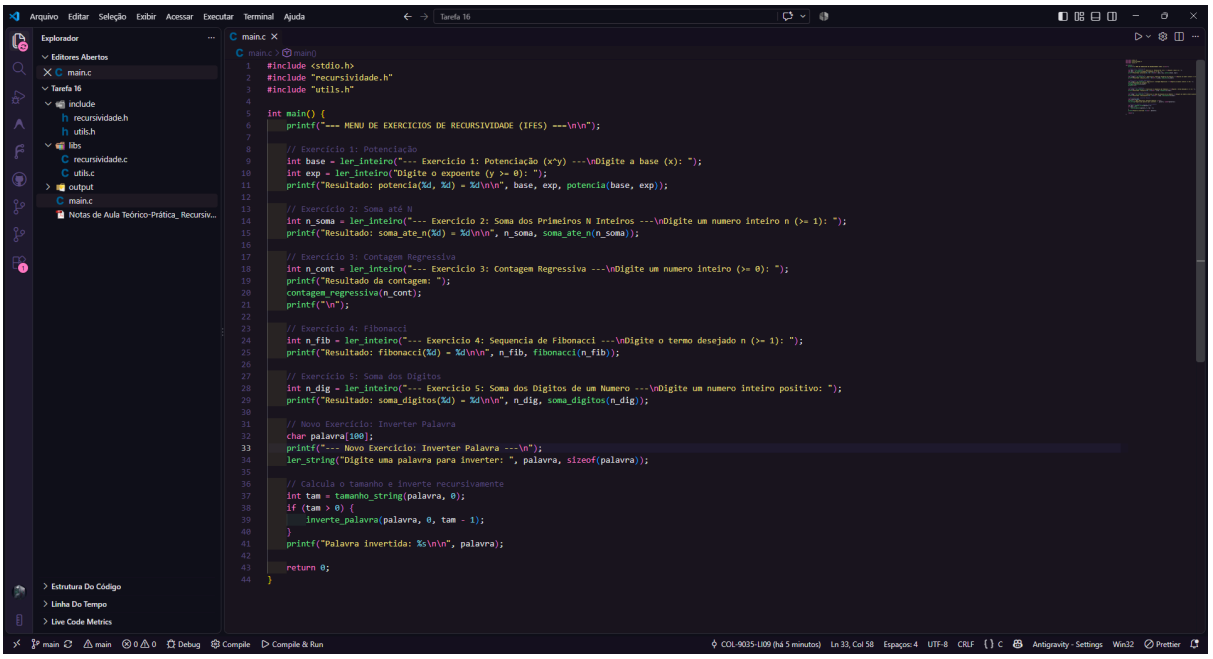
Tarefa 16 Matemática Discreta - EDUARDO MAX AMARO AMARAL

Nome: Luis Augusto de Souza



recursividade.h

```
#ifndef RECURSIVIDADE_H
#define RECURSIVIDADE_H

/**
 * EXERCÍCIO 1: Potenciação Inteira Exponencial (x^y)
 */
int potencia(int x, int y);

/**
 * EXERCÍCIO 2: Soma dos Primeiros N Inteiros Consecutivos
 */
int soma_ate_n(int n);

/**
 * EXERCÍCIO 3: Contagem Regressiva com Efeito Colateral
 */
void contagem_regressiva(int n);

/**
 * EXERCÍCIO 4: Sequência de Fibonacci
 */
int fibonacci(int n);

/**
 * EXERCÍCIO 5: Soma dos Dígitos de um Número Decimal
 */
int soma_digitos(int n);

/**
 * NOVO EXERCÍCIO: Calcula o tamanho de uma string recursivamente
 */
int tamanho_string(char *str, int index);

/**
 * NOVO EXERCÍCIO: Inverte uma palavra/string recursivamente
 */
void inverte_palavra(char *str, int inicio, int fim);

#endif
```

recursividade.c

```c
#include "recursividade.h"
#include <stdio.h>

/**
 * EXERCÍCIO 1: Potenciação Inteira Exponencial (x^y)
 */
int potencia(int x, int y) {
    // 1. Caso Base: qualquer número elevado a 0 é 1
    if (y == 0) {
        return 1;
    }
    // 2. Passo Recursivo: x^y = x * x^(y-1)
    return x * potencia(x, y - 1);
}

/**
 * EXERCÍCIO 2: Soma dos Primeiros N Inteiros Consecutivos
 */
int soma_ate_n(int n) {
    // 1. Caso Base: S(1) = 1
    if (n <= 1) {
        return 1;
    }
    // 2. Passo Recursivo: S(n) = n + S(n-1)
    return n + soma_ate_n(n - 1);
}

/**
 * EXERCÍCIO 3: Contagem Regressiva com Efeito Colateral
 */
void contagem_regressiva(int n) {
    // 1. Caso Base: atingiu o zero, imprime o zero e a mensagem de decolagem
    if (n == 0) {
        printf("0, Decolagem!\n");
        return;
    }
    // 2. Passo Recursivo: imprime o número atual e chama a função para n - 1
    printf("%d, ", n);
    contagem_regressiva(n - 1);
}

/**
 * EXERCÍCIO 4: Sequência de Fibonacci (Recorrência de 2ª Ordem)
 */
```

```
int fibonacci(int n) {
    // 1. Caso Base: F(1) = 1 e F(2) = 1
    if (n == 1 || n == 2) {
        return 1;
    }
    // 2. Passo Recursivo: F(n) = F(n-1) + F(n-2)
    return fibonacci(n - 1) + fibonacci(n - 2);
}

/**
 * EXERCÍCIO 5: Soma dos Dígitos de um Número Decimal
 */
int soma_digitos(int n) {
    // 1. Caso Base: se n for 0, a soma dos dígitos restantes é 0
    if (n == 0) {
        return 0;
    }
    // 2. Passo Recursivo: pega o último dígito (n % 10) e soma com o restante (n / 10)
    return (n % 10) + soma_digitos(n / 10);
}

/**
 * NOVO EXERCÍCIO: Função auxiliar recursiva para calcular o tamanho da string
 */
int tamanho_string(char *str, int index) {
    // 1. Caso Base: chegou ao fim da string '\0' ou quebra de linha '\n'
    if (str[index] == '\0' || str[index] == '\n') {
        return 0;
    }
    // 2. Passo Recursivo: 1 + tamanho do restante da string
    return 1 + tamanho_string(str, index + 1);
}

/**
 * NOVO EXERCÍCIO: Inverter uma palavra de forma recursiva
 */
void inverte_palavra(char *str, int inicio, int fim) {
    // 1. Caso Base: os ponteiros se cruzam ou se encontram no meio
    if (inicio >= fim) {
        return;
    }
    // Passo Intermediário / Troca: troca os caracteres das pontas
    char temp = str[inicio];
    str[inicio] = str[fim];
    str[fim] = temp;

    // 2. Passo Recursivo: chama para os caracteres internos (aproximando do centro)
    inverte_palavra(str, inicio + 1, fim - 1);
```

```
}
```

utils.h

```
#ifndef UTILS_H
#define UTILS_H

/**
 * @brief Lê um número inteiro do usuário exibindo uma mensagem personalizada.
 *        Também limpa o buffer do teclado para evitar conflitos com leituras futuras.
 * @param mensagem A string com a instrução para o usuário.
 * @return O valor inteiro digitado.
 */
int ler_inteiro(const char *mensagem);

/**
 * @brief Lê uma string/palavra do usuário com segurança.
 * @param mensagem A string com a instrução para o usuário.
 * @param dest Ponteiro para o array onde a palavra será armazenada.
 * @param tamanho O tamanho máximo do buffer.
 */
void ler_string(const char *mensagem, char *dest, int tamanho);

#endif
```

utils.c

```
#include "utils.h"
#include <stdio.h>
#include <string.h>

int ler_inteiro(const char *mensagem) {
    int valor;
    printf("%s", mensagem);
    scanf("%d", &valor);

    // Limpa o '\n' deixado pelo scanf no buffer do teclado
    int c;
    while ((c = getchar()) != '\n' && c != EOF);

    return valor;
}

void ler_string(const char *mensagem, char *dest, int tamanho) {
    printf("%s", mensagem);
    fgets(dest, tamanho, stdin);

    // Remove a quebra de linha (\n) capturada pelo fgets no final da string
    size_t len = strlen(dest);
    if (len > 0 && dest[len - 1] == '\n') {
        dest[len - 1] = '\0';
    }
}
```

main.c

```
#include <stdio.h>
#include "recursividade.h"
#include "utils.h"

int main() {
    printf("=== MENU DE EXERCICIOS DE RECURSIVIDADE (IFES) ===\n\n");

    // Exercício 1: Potenciação
    int base = ler_inteiro("--- Exercicio 1: Potenciação (x^y) ---\nDigite a base (x): ");
    int exp = ler_inteiro("Digite o expoente (y >= 0): ");
    printf("Resultado: potencia(%d, %d) = %d\n\n", base, exp, potencia(base, exp));

    // Exercício 2: Soma até N
    int n_soma = ler_inteiro("--- Exercicio 2: Soma dos Primeiros N Inteiros ---\nDigite um
numero inteiro n (>= 1): ");
    printf("Resultado: soma_ate_n(%d) = %d\n\n", n_soma, soma_ate_n(n_soma));

    // Exercício 3: Contagem Regressiva
    int n_cont = ler_inteiro("--- Exercicio 3: Contagem Regressiva ---\nDigite um numero
inteiro (>= 0): ");
    printf("Resultado da contagem: ");
    contagem_regressiva(n_cont);
    printf("\n");

    // Exercício 4: Fibonacci
    int n_fib = ler_inteiro("--- Exercicio 4: Sequencia de Fibonacci ---\nDigite o termo desejado
n (>= 1): ");
    printf("Resultado: fibonacci(%d) = %d\n\n", n_fib, fibonacci(n_fib));

    // Exercício 5: Soma dos Dígitos
    int n_dig = ler_inteiro("--- Exercicio 5: Soma dos Digitos de um Numero ---\nDigite um
numero inteiro positivo: ");
    printf("Resultado: soma_digitos(%d) = %d\n\n", n_dig, soma_digitos(n_dig));

    // Novo Exercício: Inverter Palavra
    char palavra[100];
    printf("--- Novo Exercicio: Inverter Palavra ---\n");
    ler_string("Digite uma palavra para inverter: ", palavra, sizeof(palavra));

    // Calcula o tamanho e inverte recursivamente
    int tam = tamanho_string(palavra, 0);
    if (tam > 0) {
        inverte_palavra(palavra, 0, tam - 1);
    }
```

```
    printf("Palavra invertida: %s\n\n", palavra);

    return 0;
}
```